Cross-Engine Consistency Reference

What every lesson is trying to keep consistent across all 8 engines, and why

VIDEO GAME CREATION

Purpose

This is a running, working document, not a finished reference - it will be updated every time a lesson's content changes. It exists to answer one question at a glance, per lesson: what components/techniques is this lesson trying to keep the same across every engine, and why that particular choice was made. “Why” is recorded even when the honest answer is that it's a simplification, a convenience, or an arbitrary-but-consistent choice - not every design decision in this course is deeply pedagogical, and pretending otherwise would make this document less useful, not more.

Each component below is tagged with the flavor of its rationale:

- **[Pedagogical]:** chosen specifically to teach a concept.
- **[Technical necessity]:** the engine(s) genuinely require this approach.
- **[Simplification]:** a deliberately easy stand-in for a harder "real" technique, appropriate for this point in the course.
- **[Convention]:** an arbitrary but consistently-applied choice, mostly for predictability.
- **[Consistency]:** exists specifically to keep engines aligned with each other.
- **[Robustness]:** guards against a real bug class.
- **[Practical]:** a production/classroom-logistics reason rather than a game-design one.

Entries marked FLAGGED are inconsistencies found while assembling this document - some newly discovered, some already tracked elsewhere in the project's task list and cross-referenced here. This document does not replace that task list; it's a second, lesson-indexed way to notice the same class of problem.

How To Use This Document

- Arranged by lesson number, 1 through 15 (the lessons built so far). Each entry updates as that lesson's content changes - the last-updated context lives in the surrounding project history, not in this file itself.

- Each lesson lists its Level 1/2/3 goals verbatim from that lesson's own GDD (the engine-agnostic “what”), then a Components table (the cross-engine “how,” and why it was built that way).
- A component already established in an earlier lesson and simply reused isn't re-explained in full every time it recurs - later entries point back to where it was introduced.
- For the book creation phase specifically: the Foundational Rules section below is the closest thing this course has to a single “design philosophy” statement, and the per-lesson Structure-goal notes trace one continuous throughline (separation of concerns) across the entire semester - both are likely useful raw material for a framing chapter.

Foundational Rules (apply to every lesson)

These aren't lesson-specific - they're standing decisions that shape how every lesson from 2 onward gets built, restated here because they're the load-bearing context for everything below.

Component	Purpose / Rationale	Notes
Three Levels of Accomplishment (MVP / Complete / Refined)	[Pedagogical] Gives students of very different ages and prior experience a shared, non-numeric way to know they're legitimately done, without stalling anyone or handing advanced students busy work. Established in Lesson 1, restated every week.	See Lesson_Production_Standards.docx, Rule 1.
Self-Contained Lesson Assets (no scavenger hunts)	[Practical] Protects limited lab time - any placeholder art/sound/starter file a lesson needs is pre-bundled in that lesson's Assets folder rather than sending a student searching mid-class.	See Standards Rule 3.
Visual Programming Prefers Purpose-Built Blocks	[Pedagogical] A block named for what it does (Jump To Point, Set Sprite) is self-documenting; a generic Assign Variable block isn't - using the specific block preserves the actual readability advantage visual coding is supposed to offer.	See Standards Rule 4.
Engine Deviations Require a Driving Reason	[Consistency] An engine's content may only diverge from the shared standard form for a technical deficiency or a genuine engine-norm mismatch - never because a substitute is merely "close enough." Added this term after Unreal's Lesson 2 Recolor was found using flat colors where every other engine used the real supplied sprite.	See Standards Rule 5. Directly produced the Lesson 2/3/4/13 Unreal fixes below.
Level 3's "Sound / Visual / Structure" triad shape	[Convention] Nearly every lesson from 2 through 13 offers the same three Level 3 categories (plus a lesson-specific extra, e.g. Recolor or Clamp in Lesson 2). Gives advanced students a predictable menu regardless of week or engine.	Deliberately broken in Lesson 14 - see that lesson's entry.
"Give it its own look, distinct from X" idiom	[Pedagogical] Every new object type (Bullet in L3, Enemy in L4, a row of enemies in L5) gets a Visual goal asking students to make it read as its own thing rather than a reused placeholder. Teaches basic game-feel/readability one object type at a time.	Recurs L3, L4, L5; echoed again for HUD elements from L8 onward.

Component	Purpose / Rationale	Notes
The "Structure" goal's evolving definition	[Pedagogical] Every lesson's Structure goal enforces separation of concerns, but what's being separated changes and escalates weekly: co-locate-by-trigger (L2) -> object owns its own behavior (L3/L4) -> spawner vs. behavior (L5) -> unrelated counters stay unrelated (L6/L8/L9) -> one reusable entry point, no duplicated setup (L10/L11) -> consolidate a system built piecemeal (L12) -> a purely cosmetic effect still knows nothing about game state (L13).	A full-year software-architecture throughline in miniature, not a repeated checkbox - see individual lesson notes for exact wording.

Lesson 1: Course Foundations

Whole-class introduction; no hands-on build exercise yet, so no per-engine Handout/Key/Goals table exists for this lesson (expected, not a gap - see Standards doc, File Organization).

Cross-Engine Components

Component	Purpose / Rationale	Notes
Level 1/2/3 framework introduced	[Pedagogical] Establishes the vocabulary (Minimal/Complete/Refined) used every week for the rest of the year before it's needed for real.	First use is Lesson 2's Handouts.
8-engine choice model introduced	[Practical] Lets students pick a scripting, block, or visual-coding engine while the class still runs on one shared weekly lesson.	See Engine Guide.
Capstone reveal / codename vote	[Pedagogical/Motivational] Gives the semester a concrete destination (the shooter mini-game) from week one, rather than only revealing it lesson by lesson.	See Semester_01_Capstone_GDD.docx.

Lesson 2: Programming Basics I

First hands-on build. Establishes most of the recurring Level 3 goal categories used for the rest of the semester.

Level 1/2/3 Goals (from this lesson's GDD)

- Level 1: move the sprite right via a directional button.
- Level 2: move right, left, up, and down via separate, appropriate buttons.
- Level 3 - Animate: blink while moving, static when idle.
- Level 3 - Sound: beep-beep while moving, silent when idle.
- Level 3 - Recolor: different color per direction, reset when idle.
- Level 3 - Clamp: keep movement on the screen.
- Level 3 - Structure: group everything triggered by the same action in one place.

Cross-Engine Components

Component	Purpose / Rationale	Notes
Continuous-hold-while-key-down movement	[Pedagogical] Movement runs every frame the key is held (not once per tap), matching how every commercial game handles directional input.	Unreal originally wired this to its Started pin (single-press) - corrected to Triggered (continuous) during hands-on testing.
Import the real placeholder sprite (smiley-face circles) for base look + Recolor	[Consistency, Rule 5] Recolor swaps in the actual supplied player_circle_*.png images rather than tinting one asset - matches how real 2D sprite-based games swap pre-drawn frames, and avoids teaching color-blend math this early.	Unreal originally built flat-color Materials instead of importing the real sprites - corrected this session under the new Rule 5.
Blink (visibility toggle) as "Animation"	[Simplification] A cheap stand-in for real sprite animation, which hasn't been taught yet - toggling visible/hidden on a timer is enough to read as "something is happening."	Consistent across all 8 engines.
Cooldown-gated retrigger for continuous sound	[Technical necessity] Several engines' "play sound" calls are fire-and-forget with no "is it still playing" query, so a short cooldown timer is what turns a single trigger into a believable repeating beep-beep instead of one blip.	First appearance of the cooldown-timer idiom, reused formally in Lesson 3 for fire-rate.
Screen-edge clamping	[Simplification] First bounds-checking exercise - keeps the player from wandering permanently off-screen before camera-follow or level bounds exist.	-
Structure: group by triggering action	[Pedagogical] First appearance of the Structure throughline - keep everything one key press triggers (movement, sound, recolor) together, not scattered.	See Foundational Rules above.

Flagged Inconsistencies

- **FLAGGED -** Unreal's Lesson 2 required a substantial mid-semester rebuild this session: the Blank template's default flying camera Pawn was consuming movement input, Lumen/lighting were on for a project that needs neither, and Recolor was using flat-color Materials instead of the real supplied sprite. All resolved; see this lesson's Unreal Handout/Key for the corrected Setup.

Lesson 3: Programming Basics II

Introduces spawned, self-managing objects (the bullet) - the course's first object that isn't the player.

Level 1/2/3 Goals (from this lesson's GDD)

- Level 1: one key press spawns one projectile that flies upward and disappears off-screen.

- Level 2: repeated firing produces independent projectiles, capped by a fire-rate cooldown.
- Level 3 - Sound: play a sound the moment a projectile is fired.
- Level 3 - Visual: give the projectile its own look, distinct from the player.
- Level 3 - Structure: keep a spawned object's own behavior with the object itself.

Cross-Engine Components

Component	Purpose / Rationale	Notes
Discrete spawn-on-press (vs. L2's continuous hold)	[Pedagogical] Deliberately contrasts with Lesson 2's continuous movement - firing is a one-shot event per press, teaching the difference between event-driven and continuous/polled input handling.	-
Fire-rate cooldown, formalized as a Level 2 requirement	[Technical necessity + Pedagogical] Extends Lesson 2's informal sound-cooldown idiom into a named, required mechanic: without it, holding the fire key would flood the screen with projectiles.	-
Spawned-object self-cleanup (off-screen destroy)	[Pedagogical] First "an object manages its own lifecycle" pattern - the bullet checks its own position and destroys itself, rather than something else tracking and culling it.	-
"Give it its own look" (Visual goal)	[Pedagogical] First appearance of this recurring idiom (see Foundational Rules) - here, using the real bullet_yellow.png sprite already makes this true by default in every engine.	Unreal's Visual idea originally re-tinted the bullet's own color, which was invisible against a sprite already showing that color - corrected to a scale-pulse flourish this session.
Structure: object owns its own behavior	[Pedagogical] The spawner (Level Blueprint / equivalent) is limited to spawning + cooldown + sound; movement and cleanup live entirely inside the bullet itself.	-

Lesson 4: Programming Basics III

Introduces collision/overlap between two distinct object types for the first time.

Level 1/2/3 Goals (from this lesson's GDD)

- Level 1: a stationary enemy is destroyed, along with the projectile, on contact.
- Level 2: destruction is clean and exactly-once even with rapid-fire hits; a miss still behaves exactly as before.
- Level 3 - Sound: play a sound the moment a projectile hits the enemy.
- Level 3 - Visual: give the enemy its own look, distinct from the player and the projectile.
- Level 3 - Structure: keep an object's own hit-response with that object itself.

Cross-Engine Components

Component	Purpose / Rationale	Notes
Cross-object collision detection	[Pedagogical] First lesson where two independently-built object types (bullet, enemy) have to detect and react to each other.	-
"Destroy is safe to call twice" idempotency	[Robustness] Explicitly taught rather than assumed - two bullets overlapping the same enemy on one tick must not error.	-
"Give it its own look" (Visual goal, 2nd appearance)	[Pedagogical] Same idiom as Lesson 3, now for the enemy - reinforces the pattern on a second object type before Lesson 5 multiplies it.	Unreal's enemy Material originally used a flat tint instead of the real enemy_invader_red.png sprite - corrected this session; this fix is also what makes Lesson 5's stripe idea and Lesson 13's flash idea work correctly downstream.
Structure: hit-response stays with the object hit	[Pedagogical] The enemy's own overlap event does the Cast, both Destroy calls, and the sound - the bullet needs zero new code this week.	-

Lesson 5: Programming Basics IV

Multiplies the enemy from Lesson 4 via a loop - the first real test of whether Lesson 3/4's "object owns its own behavior" discipline actually pays off.

Level 1/2/3 Goals (from this lesson's GDD)

- Level 1: several enemies appear in a row via a loop; shooting any one destroys just that one.
- Level 2: every enemy in the row is independently shootable with no errors; a miss still behaves as before.
- Level 3 - Sound: already covered in Lesson 4 - confirm it still plays correctly per enemy.
- Level 3 - Visual: give enemies row-to-row variety so they don't all look identical.
- Level 3 - Structure: the spawn loop only creates and positions enemies; hit-response stays with each enemy.

Cross-Engine Components

Component	Purpose / Rationale	Notes
Loop-based spawning (vs. one-at-a-time placement)	[Pedagogical] First "spawn N instances from one loop" pattern - reused conceptually by Lesson 11's wave respawn.	-

Component	Purpose / Rationale	Notes
Regression check as an explicit Level 3 goal	[Pedagogical] The Sound goal is literally "confirm Lesson 4 still works" rather than new work - makes the payoff of L3/L4's Structure discipline visible: nothing needed to change for sound to keep working across N instances.	First appearance of this framing; the pattern (new lesson silently re-validating old work) recurs informally elsewhere but isn't always named this explicitly.
Per-instance tint for row variety	[Pedagogical] First real use of a per-instance Color/tint parameter driven by loop index - directly depends on each engine's enemy Material/sprite already supporting a genuine tint (not a flat color).	Unreal: works correctly with zero changes to this lesson's own content, once Lesson 4's Material was fixed to a real Texture x Color-multiply setup this session.

Lesson 6: Programming Basics V

First win condition. Per Lesson 15's own retrospective, this lesson is the semester's declared Level 1 (MVP) milestone.

Level 1/2/3 Goals (from this lesson's GDD)

- Level 1: "You Win!" appears the instant the last enemy is destroyed.
- Level 2: the win message fires exactly once, exactly at zero - no early or repeated triggers.
- Level 3 - Sound: play a victory sound at the win condition.
- Level 3 - Visual: make the win message feel like a celebration, not plain text.
- Level 3 - Structure: keep the win-counter and its zero-check together, separate from hit-detection.

Cross-Engine Components

Component	Purpose / Rationale	Notes
Win-condition detection (count reaches zero)	[Pedagogical] First state-transition condition in the course, distinct from the continuous per-frame/per-event logic used so far.	-
"State-transition" sound (vs. action/reaction sound)	[Pedagogical] A third sound category alongside L2/L3's action sounds and L4's reaction sound - fires once on entering a new game state.	-
Structure: unrelated counters stay unrelated	[Pedagogical] Win-counter logic and hit-detection logic both touch "an enemy was destroyed," but are kept in separate places - first explicit case of two systems sharing a trigger without merging.	Named again explicitly in Lesson 8's own GDD text ("the same separation principle used since Lesson 6").

Component	Purpose / Rationale	Notes
Semester milestone marker: Level 1 MVP	[Structural] Lesson 15 explicitly retroactively labels this lesson's complete playable loop as the semester's Level 1 checkpoint.	See Lesson 15 below.

Lesson 7: Core Mechanics I

First autonomous (non-player-input) continuous motion, and the first object with three possible fates (shot, escaped, or never spawned).

Level 1/2/3 Goals (from this lesson's GDD)

- Level 1: every enemy moves slowly toward the player; one that reaches the bottom unshot is removed and does not count as a win.
- Level 2: a moving enemy can still be shot correctly; an escaped enemy never falsely triggers "You Win!"
- Level 2 - Bugfix (GameMaker / 001 Game Creator only): win-counting moved off a generic "destroyed" hook onto the specific hit-detection code, since enemies can now be removed two different ways.
- Level 3 - Visual: vary descent speed instead of perfect lockstep.
- Level 3 - Structure: keep movement/escape-cleanup and hit-detection/win-counting in clearly separate places.

Cross-Engine Components

Component	Purpose / Rationale	Notes
Autonomous continuous motion	[Pedagogical] Reuses Lesson 2's delta-time-scaled movement technique, but now driving an NPC instead of reading player input - shows the same technique generalizes.	-
"Escape" as a second removal path	[Pedagogical] Sets up the hit-vs-escape separation that recurs formally through Lessons 8 and 9.	-
Documented, reasoned engine deviation (GameMaker / 001GC bugfix)	[Technical necessity] A genuine, already-self-documented example of Rule 5's bar being met: these two engines' generic "destroyed" hook fires for both hit AND escape, so win-counting has to move to the more specific hit-detection code, not a cosmetic choice.	Good reference example when evaluating whether a future deviation meets the Rule 5 bar.

Resolved

- **RESOLVED -** Unreal's BP_Enemy movement used the same Add Actor World Offset + Delta Location node that required "Split Struct Pin" in Lessons 2 and 3 before a value could be wired into just one axis, but this lesson's instructions didn't mention that step. Found while building this document, fixed the same day in the Handout and Key. The broader audit across Lessons 5, 6, 8, 9, 11, and 14 (Task #254) is still open.

Lesson 8: Core Mechanics II

Introduces a second scarce resource (lives) alongside the win-condition enemy count.

Level 1/2/3 Goals (from this lesson's GDD)

- Level 1: an escaped enemy costs one life (starting at 3); "Game Over" appears at zero.
- Level 2: shooting never costs a life, and an escape is never counted as a kill - the two counters stay fully independent.
- Level 3 - Visual: a live, continuously-updating lives readout.
- Level 3 - Sound: a distinct sound when a life is lost.
- Level 3 - Structure: keep lives/escape logic and kills/hit-detection logic separate - the same separation principle used since Lesson 6.

Cross-Engine Components

Component	Purpose / Rationale	Notes
Second independent counter (lives, alongside score/kills)	[Pedagogical] First lesson with two scarce resources that must never cross-contaminate each other's logic.	-
First live HUD element	[Pedagogical] First on-screen number that must stay continuously accurate - a soft preview of Lesson 9's formal HUD requirement.	Formalized in Lesson 9.
Structure: explicitly named continuity with Lesson 6	[Consistency] The lesson's own GDD text states this is "the same separation principle used since Lesson 6" - a rare case where the throughline is self-documented in the shipped materials rather than only visible from the outside.	-

Lesson 9: Core Mechanics III

Formalizes the HUD pattern only informally previewed in Lesson 8, and adds score as a third tracked value.

Level 1/2/3 Goals (from this lesson's GDD)

- Level 1: destroying an enemy adds 10 points; score and lives are both always visible.
- Level 2: score and lives readouts never show a stale or wrong number - they always match reality immediately.
- Level 3 - Visual: style the HUD for a more game-like look, and/or add scoring variety.
- Level 3 - Structure: keep all display-writing code in one place, separate from the logic that changes score/lives - a new kind of separation, distinct from the hit-vs-escape separation used since Lesson 6.

Cross-Engine Components

Component	Purpose / Rationale	Notes
Single-source-of-truth HUD update function	[Pedagogical] The UI is a read-only reflection of internal variables, written from one place, rather than being set ad hoc wherever a value changes - directly prevents a real, common "stale UI" bug class.	First formal appearance; Lesson 11 extends it to the wave number.
Structure: display-writing vs. game-logic separation	[Pedagogical] Explicitly named in the lesson's own GDD text as "a new kind of separation ... distinct from the hit-vs-escape separation used since Lesson 6" - the throughline deliberately reframes itself for a new kind of concern this week rather than repeating the same lesson.	-

Lesson 10: Core Mechanics IV

First win/lose freeze state and the first restart-to-known-good-state requirement.

Level 1/2/3 Goals (from this lesson's GDD)

- Level 1: on win or lose, the game freezes (score/lives stop, hits/escapes do nothing); pressing R resets and starts a new round.
- Level 2: a restart is a genuinely clean slate - no leftover enemies or stale score/lives, no matter how many times replayed.
- Level 3 - Visual: a transition effect (fade, flash, or similar) on end/reset.
- Level 3 - Structure: one reset routine, called from both first game start and every restart - no setup logic duplicated anywhere.

Cross-Engine Components

Component	Purpose / Rationale	Notes
Freeze-on-end state	[Pedagogical] First lesson requiring the game to stop reacting to input/events on purpose, rather than always being "live."	-
Single reset routine, multiple call sites (DRY)	[Pedagogical] First explicit "don't duplicate setup logic" requirement - the same routine must run identically whether it's the very first game start or the fifth restart.	Directly extended by Lesson 11, which adds a third call site (wave clear) to the same principle.

Lesson 11: Core Mechanics V

Extends Lesson 10's single-reset-routine principle to a second, harder case: reusing the same spawn logic for wave progression.

Level 1/2/3 Goals (from this lesson's GDD)

- Level 1: clearing a wave spawns a new wave (up to 3 total); the current wave number is shown on screen.
- Level 2: wave transitions are seamless and correct every time; the right win/lose state still triggers on the final wave; restarting always returns to wave 1.
- Level 3 - Visual: a brief on-screen wave announcement.
- Level 3 - Structure: enemy-spawning code lives in one reusable place, called from first game start, every restart, AND every wave clear - no spawning logic duplicated anywhere.

Cross-Engine Components

Component	Purpose / Rationale	Notes
Wave-number HUD element	[Pedagogical] Second application of Lesson 9's single-source-of-truth HUD pattern to a new value.	-
Single spawn routine, three call sites (DRY, escalated)	[Pedagogical] Directly extends Lesson 10's single-reset-routine requirement - now three call sites (first start, restart, wave clear) must all funnel through the same code, not two.	-

Lesson 12: Hearing It Happen

A consolidation lesson: audits and unifies four sound cues built piecemeal across five prior lessons (Lessons 3, 4, 6/10, 11) rather than introducing a new mechanic.

Level 1/2/3 Goals (from this lesson's GDD)

- Level 1: four sounds (shot, hit/explosion, wave-clear, game-over) each play at the correct moment at least once.
- Level 2: all four sounds fire reliably every time with no missing/duplicate playback, and none interfere with each other.
- Level 3 - Audio: add the optional victory sound from Lesson 6 if not already built - it slots into the same code path as wave-clear, opposite outcome.
- Level 3 - Structure: every sound call lives physically inside the branch representing its event - none in a generic "every frame" check.

Cross-Engine Components

Component	Purpose / Rationale	Notes
Cross-lesson audio consolidation pass	[Consistency] This lesson's entire purpose is auditing prior weeks' work for consistency and collisions - thematically the closest any single lesson comes to what this reference document itself is doing.	-

Component	Purpose / Rationale	Notes
Structure: sound calls live inside their own event branch	[Pedagogical] Ties every prior lesson's individual "sound on this specific event" pattern (L2 movement, L3 fire, L4 hit, L6 win, L11 wave-clear) into one named principle.	-

Lesson 13: A Hit That Reacts

Adds a hit-flash and an independent explosion effect. Heavily reworked this session after discovering the same cross-engine "tint can't produce a true white flash" trap in multiple engines' original content, Unreal included.

Level 1/2/3 Goals (from this lesson's GDD)

- Level 1: every hit briefly flashes the enemy and spawns an independent explosion; score/lives/wave/win-lose logic still resolve at exactly the same moments as before.
- Level 2: flash and explosion play correctly on every hit, with no enemy stuck mid-flash and no leftover explosions; game-state timing is provably unaffected.
- Level 3 - Visual: smoother color fades, a scaling/animated explosion, or a multi-frame flicker instead of a flat flash.
- Level 3 - Structure: the explosion effect is fully self-contained - it knows nothing about score, lives, or waves, the same discipline bullets and enemies already follow.

Cross-Engine Components

Component	Purpose / Rationale	Notes
Delay/pause-mid-script sequencing	[Technical necessity] "Flash, then a beat, then remove" needs the ability to pause mid-function - Blueprint's Delay, Godot's await, and Unity's Coroutine all do this directly; GameMaker and TIC-80 can't pause mid-script at all and need a structurally different two-step (alarm/counter) approach instead.	A genuine, documented case of language capability shaping implementation, not just cosmetic engine differences.

Component	Purpose / Rationale	Notes
Real sprite/Material swap for the flash (not a color tint)	[Technical necessity + Consistency] A multiply-based color tint can only darken/mix with a sprite's existing color - tinting an already-red enemy toward white is invisible. Every engine swaps in the real supplied enemy_invader_white.png (or, for Unreal, a Material built from it) instead.	Unreal originally used Set Vector Parameter Value("Color", White) against a Material whose Color parameter already defaulted to White - a textbook no-op, corrected this session to a genuine Set Material swap. Godot/Unity/GameMaker's own instructions already warned against this exact trap; Unreal's content just hadn't been checked against the same standard until Rule 5 existed.
Self-contained explosion effect	[Pedagogical] Extends the "object owns its own behavior" principle (L3/L4/L5) to a purely cosmetic, transient effect - it spawns, animates itself on a timer, and destroys itself with zero help from the enemy or spawner.	Unreal's BP_Explosion originally used a plain colored Cube/Sphere instead of the real explosion_placeholder.png sprite - corrected this session to a textured Plane Material under Rule 5.

Flagged Inconsistencies

- **FLAGGED -** GameMaker's explosion effect has a known scale-origin risk not yet resolved - tracked separately as Task #243 ("Lesson 13 GameMaker: fix explosion scale origin risk"), still open as of this document's first pass.

Lesson 14: Raising the Stakes

First data-driven difficulty scaling, and the first deliberate departure from the Sound/Visual/Structure Level 3 template used every lesson since Lesson 2.

Level 1/2/3 Goals (from this lesson's GDD)

- Level 1: enemy count increases by one each wave (5, 6, 7), using a single formula referenced everywhere it's needed - no more, no less.
- Level 2: enemy speed also increases each wave, on top of the growing count; restarting correctly returns everything to wave 1's original values.
- Level 3: choose exactly ONE optional stretch feature (destructible shields, a power-up, a bonus enemy, formation-marching movement, or a title/game-over screen) and make a good-faith attempt at it.

Cross-Engine Components

Component	Purpose / Rationale	Notes
Parameterized difficulty (shared formula, not per-wave hardcoding)	[Pedagogical] First explicit "don't hardcode, derive from one formula" requirement - a data-driven design principle rather than a code-organization one.	-
Level 3 format change: single free-choice feature	[Pedagogical, deliberate] Breaks the fixed Sound/Visual/Structure trio used every lesson from 2 through 13 in favor of one open-ended choice among five options - a deliberate scaffolding-removal moment now that students have a full semester's foundation.	Worth confirming in the book manuscript that this shift is called out explicitly as intentional, not an inconsistency with the rest of the semester.

Lesson 15: Ready to Show

Semester close. No new mechanic - a retrospective lesson that re-maps the weekly Level 1/2/3 framework onto the entire semester's arc, plus a bridge to Semester 2.

Level 1/2/3 Goals (from this lesson's GDD)

- Level 1 MVP (Lesson 6) reached: a complete, playable loop - move, shoot, score, lives, a real win and lose state.
- Level 2 Complete (Lesson 11) reached: real depth - multiple waves, sound, and hit reactions all working together.
- Level 3 Refined (today) reached: difficulty scaling is present, and at least a good-faith attempt at one chosen stretch feature exists.

Cross-Engine Components

Component	Purpose / Rationale	Notes
Weekly framework applied at the project level	[Pedagogical] The same Level 1/2/3 rubric used every single week for one exercise is shown to also describe the entire 14-lesson arc (MVP = Lesson 6, Complete = Lesson 11, Refined = today) - the semester's "zoom out" moment.	-
"What's Coming When You Get Back" bridge section	[Pedagogical] Previews Semester 2 content so the year-long arc doesn't feel like it dead-ends at the winter break.	See Task #207.

Open Items Index

Every FLAGGED item above, gathered in one place for quick triage. Cross-references the project task list where one already exists.

- Lesson 7 (Unreal): RESOLVED - the missing “Split Struct Pin” step was added to the Handout/Key. (Task #254's broader audit is still open for Lessons 5, 6, 8, 9, 11, 14.)
- Lesson 13 (GameMaker): explosion effect has a known scale-origin risk, not yet fixed. (Task #243.)
- Lesson 14: the Level 3 format change (single free-choice feature, breaking the Sound/Visual/Structure template) is intentional, but should be called out explicitly as intentional wherever this course's lessons are described in prose (e.g. the book manuscript), so it doesn't read as a template the writer forgot to follow.
- General: Task #119 (rebalance pacing if the Lessons 3-15 buildout revealed issues) and Task #249 (eventually QA-validate all engines in all modes, not just the current focus subset) remain open and may surface further inconsistencies for this document once resolved.

Revision Note

First pass, written after Lessons 1-15 (Semester 1) were complete and shortly after Unreal's Lesson 2/3/4/13 content was rebuilt under the new Engine Deviation rule. Update this document alongside any future lesson content change - new component, changed rationale, or newly discovered inconsistency - the same way Lesson_Production_Standards.docx is kept current.